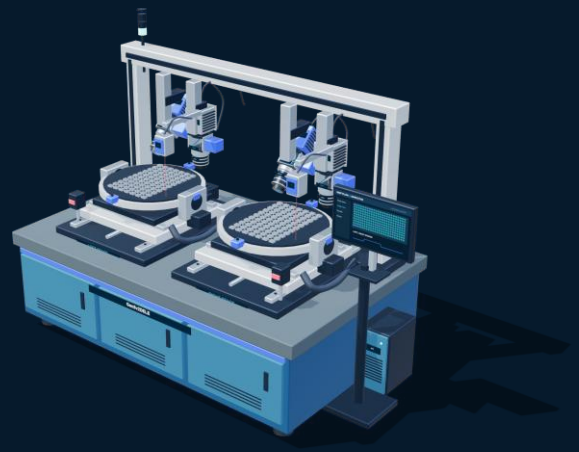
SwiftFlow + HawkeyePlus

工业检测与测量标准化平台

架构、流程与协同交付



SwiftFlow

设备、工艺与现场执行

+



HawkeyePlus

数据、模型与研发验证

从硬件设计到算法研发与现场应用

2026.10

汇报目录

先建立统一工程方法，再展开两个平台及其协同交付

01



总体概念与建设目的

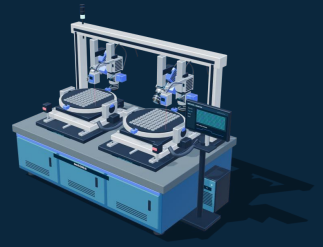
建立统一的工程方法

平台关系

建设目的

03–06

02



SwiftFlow

设备、流程与算法执行

设备接入

流程设计

07–10

03



HawkeyePlus

工业视觉模型研发与验证

模型研发流程

当前模型能力

11–16

04



协同交付

从研发效果到现场可用

适配与验收

统一交付资产

17–18

最后：总结与交流 19–20

01 总体概念与建设目的

建立统一的工程方法

如何让硬件、工艺与算法形成统一方法？



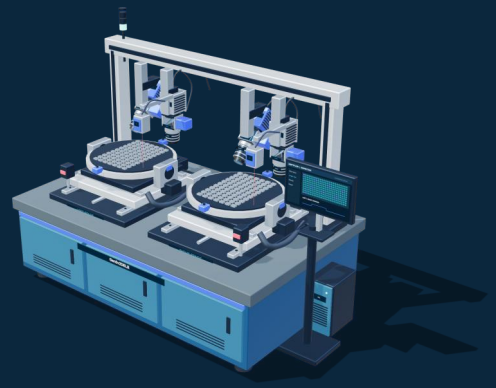
01.1	01.2	01.3
平台关系	建设目的	标准流程

SwiftFlow + HawkeyePlus｜一套标准方法，两个互补平台

一套标准流程，贯通检测与测量

以 SwiftFlow 与 HawkeyePlus 协同，连接硬件设计、检测流程与算法研发

SwiftFlow



- 设备接入
- 流程编排
- 现场执行

把硬件、算法和工艺组织成可执行流程

样本 →

← 模型包

HawkeyePlus



- 数据与标注
- 模型训练
- 验证与发布

把工业图像沉淀为可验证、可交付的模型

共同方法：明确目标 → 设计实现 → 适配验证 → 标准交付



检测

缺陷位置 · 类型 · 判定



测量

尺寸 · 高度 · 几何指标



可追溯交付

配置 · 版本 · 验收结果

为什么要建立统一的工程方法

让参与者知道每一步需要什么、交付什么，以及如何判断完成

减少衔接歧义



硬件、成像与算法各有约束

统一设计输入

- 检测 / 测量目标
- 成像质量与标定要求

减少重复搭建



成熟经验应成为可复用资产

统一交付资产

- 设备与流程配置
- 模型与算法参数

减少交付不确定性



研发效果需要在现场条件下复现

统一验收口径

- 独立样本与参考结果
- 效果、节拍与结果留痕

标准化的价值：让项目交接、方案复用与后续维护有共同依据

形成工程方法与交付约定，再由两个平台承载具体能力

从工程目标到标准化交付

推荐标准流程：每一步都有明确输入、负责内容和可检查的交付物

01 明确目标



检测什么 / 测量什么

质量、节拍与约束

需求与验收基线

02 硬件与成像



相机、镜头、光源

运动、测高与标定

硬件方案与样本

03 检测流程



定位、采集与时序

异常路径与结果组织

流程与设备配置

04 算法开发

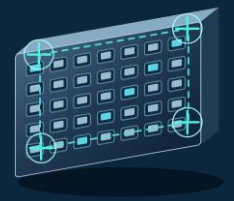


传统视觉 / 几何测量

AI 训练与验证

算法参数 / 模型包

05 联合验证



现场样本与参考值

效果、测量与节拍

验证记录与结论

06 标准交付



配置、模型与版本

结果归档与迭代

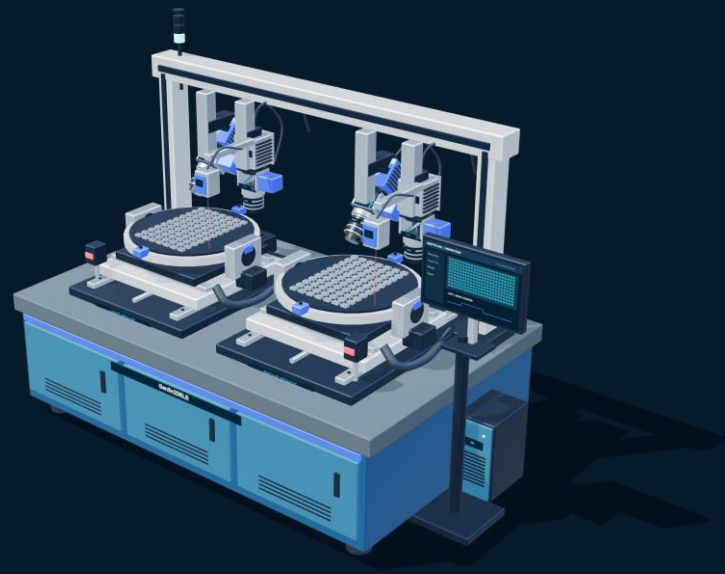
可复用工程资产

SwiftFlow	承载设备接入、调试标定、流程执行与结果输出
HawkeyePlus	承载训练数据、模型研发、评估验证与版本管理

共同交付：硬件基线 + 流程基线 + 算法基线 + 验收证据

02 SwiftFlow

设备、流程与算法执行



如何把设备与算法组织成可执行流程？

02.1
设备接入

02.2
流程设计

02.3
检测与测量

SwiftFlow + HawkeyePlus | 一套标准方法，两个互补平台

SwiftFlow | 设备与流程执行平台

用可配置的流程，把设备、传统视觉、模型推理和结果组织成执行链

流程设计



可视化编排

工艺与子流程复用

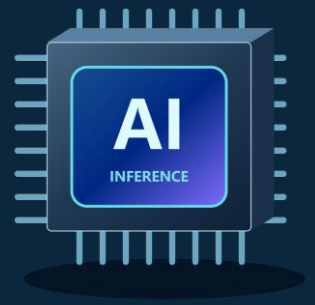
统一执行引擎



串并行协同 · 共享数据

运行控制 · 异常处理

算法与结果



传统视觉 + 模型推理

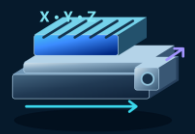
测量计算 + 结果归档

统一设备接入

标准接口 · 多实例 · 厂商适配



工业相机



精密运动



受控照明



激光测高



光学 / 测量



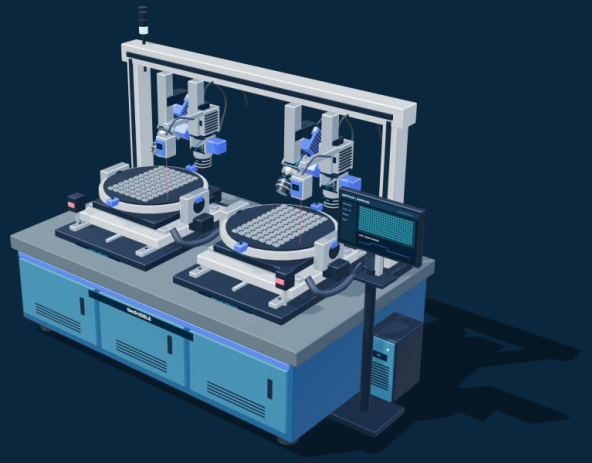
IO / 产线通信

全链路支撑：日志告警 · 运行计时 · 结果追溯 · 存储管理 · 仿真调试

从硬件方案到可执行检测流程

将硬件选型、设备调试和工艺经验，转化为可复用的配置与流程

工程设计输入



相机 / 镜头 / 光源 / 运动 / 传感器

明确视野、分辨率、节拍与标定要求

设备配置与流程实现



设备映射

调试与标定

参数与流程复用



定位 → 采集 → 分析 → 判定与记录

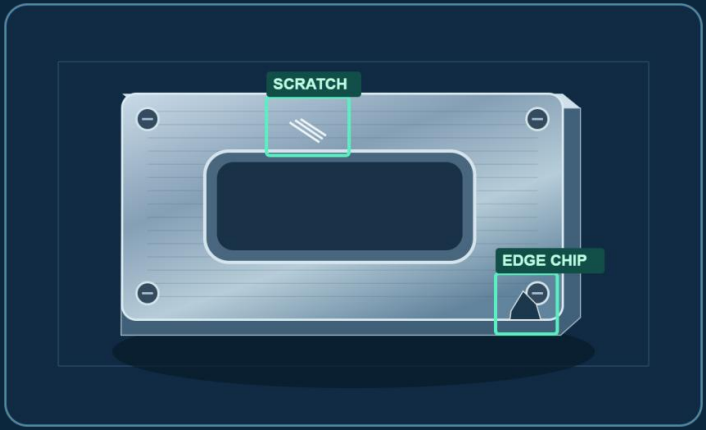
交付的是可复用的工程资产

设备映射与配置 · 标定参数 · 检测流程 · 工艺参数 · 结果定义

以同一套流程，实现检测或测量

传统视觉、AI 与测量算法按任务组合，共享采集、编排和结果追溯能力

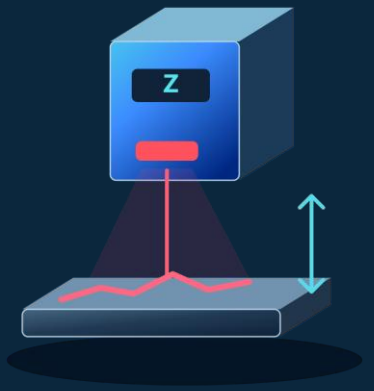
检测：找到缺陷并判定



图像 → 传统视觉 / AI → 判定

输出：位置、类别与合格判定

测量：获得定义明确的指标



标定 → 采集 / 分割 → 测量计算

输出：尺寸、高度与几何指标

共同底座：数据采集 → 算法执行 → 规则判定 → 结果归档

测量结果需要标定、单位与参考验证；AI 可以辅助定位或分割

03 HawkeyePlus

工业视觉模型研发与验证

如何从数据出发，形成可验证、可交付的模型？



03.1
模型研发流程

03.2
当前模型能力

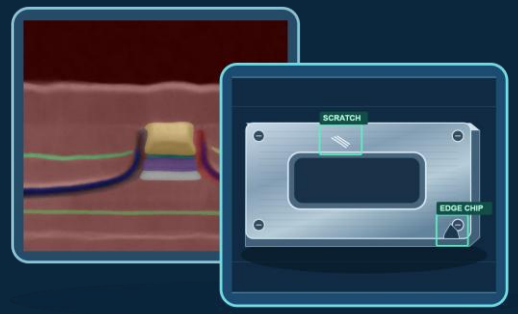
03.3
验证与版本管理

SwiftFlow + HawkeyePlus｜一套标准方法，两个互补平台

HawkeyePlus | 工业视觉模型研发

从工业图像到可交付模型，贯通数据、训练、验证与发布

01 数据与标注资产



工业图像 + 标签 / 掩码

上传导入 | 标注转换 | 样本预览

可视化训练中枢

数据集配置 | 模型配置 | 训练参数



任务编排 | GPU 选择 | 训练监控

02 验证与模型交付

评估与推理
指标评估 · 结果预览

推理模型导出
模型转换 · 打包下载

模型版本发布
按产品管理 · 激活切换

多框架训练能力

PaddlePaddle
检测 · 分类 · 分割

PyTorch
检测 · 分割

anomalib
无监督异常检测

运行与资产底座

算力与环境
GPU / Python

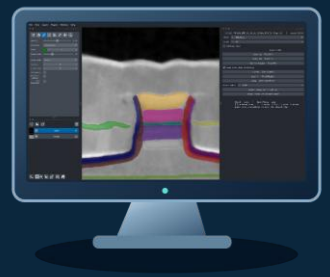
数据与资产
图像 · 权重 · 日志

- **统一管理** 项目归属 · 用户权限 · 环境检查 · GPU 状态 · 存储配额 · 操作记录

从数据到模型的可视化闭环

将分散的准备、训练与验证环节，组织为统一的模型研发工作流

01 准备数据



导入图像与标注

转换格式 · 检查样本

02 组合配置



数据 + 模型 + 参数

模板复用 · 可视化配置

03 训练监控



启动任务 · 查看曲线

进度 · 损失 · 评估指标

04 验证效果



独立评估 · 图像推理

预览结果 · 判断可用性

05 模型交付



导出下载 · 注册版本

按产品切换激活模型

依据验证结果，迭代数据与配置

四类视觉任务，共用一套平台能力

目标检测

发现缺陷与目标位置

图像分类

识别类别与质量状态

语义分割

刻画区域与结构边界

无监督异常检测

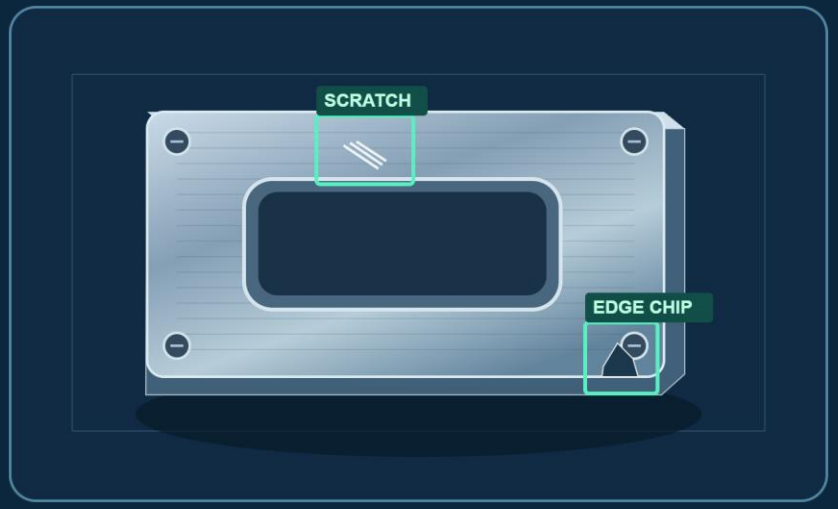
利用正常样本学习异常

降低使用门槛 / 复用训练经验 / 沉淀模型资产

Detection | 工业缺陷检测

目标框、类别与置信度，让缺陷位置与类型一目了然

缺陷定位与识别



输出：目标框 · 类别 · 置信度

检测结果示意，非真实模型预测

PaddleDetection

7 presets

PP-YOLOE · YOLOv3 · PicoDet · RT-DETR

Faster R-CNN · RetinaNet · CenterNet

TorchDet

4 presets

Faster R-CNN · RetinaNet · FCOS · SSD / SSDLite

SSD：VGG16 → SSD300；MobileNetV3 → SSDLite320



COCO 数据与标注

框标注 · 类别管理



训练与指标跟踪

mAP / mAP50 · 训练曲线



检测结果预览

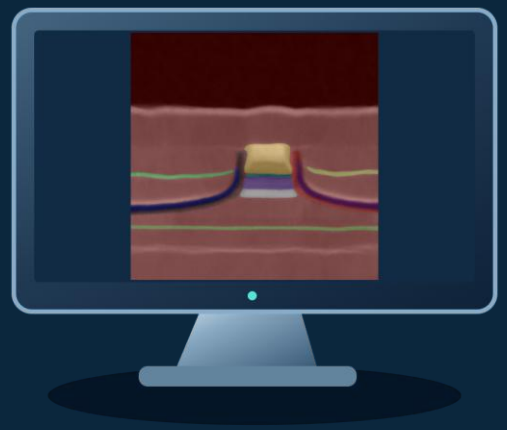
定位缺陷 · 查看类别

预设按框架区分；PP-YOLOE、RT-DETR 等不属于 TorchDet。

分割模型 | 像素级结构与区域识别

以 TEM 分割示例说明任务；模型覆盖轻量网络、编码器解码器与 Transformer

像素级预测示例



图像区域 · 结构边界 · 标签掩码

项目内 TEM 分割示例

PaddleSeg

7 presets

PP-LiteSeg · BiSeNetV2 · OCRNet · UNet · DeepLabV3+
SegFormer-B0 · FCN

TorchSeg

4 presets

UNet · DeepLabV3 (UI: DeepLabV3+) · FCN · LR-ASPP

TorchSeg 的 DeepLabV3P 使用 torchvision DeepLabV3，并非 V3+ 解码器。



数据组织

图像 + 掩码 + 训练 / 验证划分



评估指标

mIoU · Dice · 准确率



业务用途

缺陷区域 · 微观结构 · 边界分析

分类与异常检测｜补齐视觉任务覆盖

从整图类别判断，到基于正常样本的异常识别，复用同一套模型研发流程

PaddleClas｜图像分类

按类别学习，输出整图分类结果



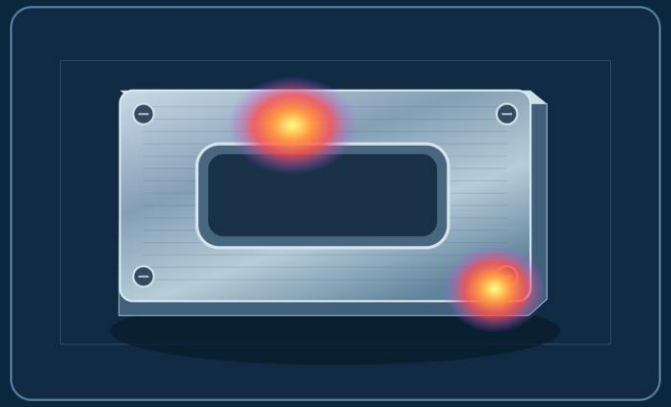
分类输出示意

9 个分类变体

ResNet18 · ResNet34 · ResNet50 · ResNet101
MobileNetV3 Small / Large (x1.0)
PP-LCNet x1.0 · PP-HGNet Small
Swin Transformer Tiny (patch4 / window7 / 224)

TorchAnomaly｜异常检测

基于正常样本，输出异常分数与热力图



异常热力图示意

5 个 anomalib 预设

PatchCore · PaDiM · STFPM · EfficientAD

SuperSimpleNet

仅列当前界面预设；自定义 YAML 的兼容性取决于所选后端。

04 协同交付

从研发效果到现场可用



如何用交付包与联合验证连接两个平台？

04.1
适配与验收

04.2
统一交付资产

04.3
结果与版本追溯

SwiftFlow + HawkeyePlus | 一套标准方法，两个互补平台

以标准交付包，衔接研发与现场

训练成果经过适配与联合验证，再进入现场检测或测量流程

HawkeyePlus



验证后的模型与版本

→

适配与联合验收

- **输入一致**
  图像尺寸、通道与预处理
- **输出匹配**
  标签、阈值与结果解释
- **现场验证**
  独立样本、测量参考与节拍

→

SwiftFlow



流程中的推理与判定

建议统一交付资产



硬件与标定

配置 · 标定 · 参考



检测流程

步骤 · 参数 · 规则



算法与模型

版本 · 标签 · 阈值



验收与追溯

样本 · 结论 · 记录

把能力连接成可复用的标准流程

用共同的方法组织工程工作，以两个平台承载能力，以交付基线保证衔接

统一工程方法



从检测或测量目标出发

明确硬件、流程与算法的输入输出

两个平台协同



SwiftFlow 承载设备与流程执行

HawkeyePlus 承载模型研发与验证

形成标准交付



配置、模型与验收证据共同交付

适配验证后进入现场，保留版本与结果

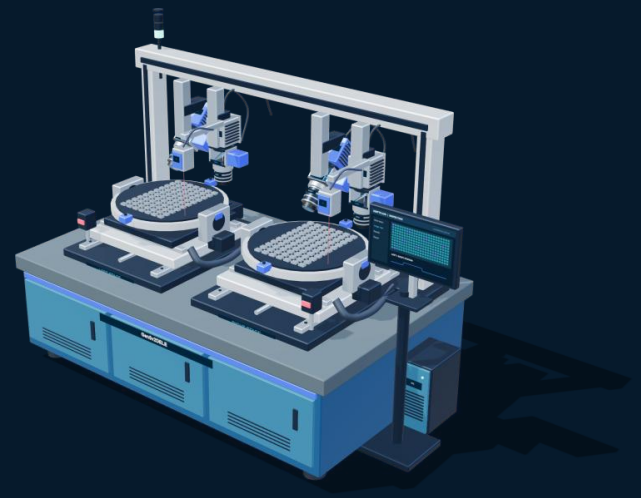
目标明确 → 工程实现 → 联合验证 → 可追溯交付

最终目标：方便用户以一致的方法，实现检测或测量目的

谢谢

讨论与交流

从硬件设计到检测与测量交付



- 如何确定首个应用场景？
- 如何定义验收与交付基线？
- 哪些流程、模型与经验可以优先复用？



一套标准方法 · 两个互补平台 · 可复用的工程交付